

Guide Utilisateur claude-socle

Maîtrisez Claude Code avec un workflow professionnel

claude-socle

Janvier 2025

Table des matières

1	Introduction	4
1.1	Qu'est-ce que claude-socle ?	4
1.2	Pourquoi utiliser claude-socle ?	4
1.3	Prérequis	4
2	Installation	5
2.1	Option 1 : Script d'installation (recommandé)	5
2.2	Option 2 : Installation manuelle	5
2.3	Option 3 : Création d'un nouveau projet	5
2.4	Vérification de l'installation	6
3	Le Workflow Explore → Plan → Code → Commit	7
3.1	Vue d'ensemble	7
3.2	Étape 1 : EXPLORE	7
3.2.1	Commande	7
3.2.2	Quand l'utiliser ?	7
3.2.3	Exemple pratique	7
3.3	Étape 2 : PLAN	8
3.3.1	Commande	8
3.3.2	Quand l'utiliser ?	8
3.3.3	Exemple pratique	8
3.4	Étape 3 : CODE	9
3.4.1	Commandes disponibles	9
3.4.2	Exemple TDD	9
3.5	[GREEN] GREEN - Implémentation minimale	9
3.6	[REFACTOR] REFACTOR - Amélioration	10
3.7	Changements détectés	10
3.7.1	Fichiers modifiés (4)	10
3.7.2	Fichiers de test (2)	10
3.8	Message de commit proposé	11
3.9	Étape 1/5 : Création de la branche	11
3.10	Étape 2/5 : Exploration	11
3.11	Étape 3/5 : Planification	11

3.12	Étape 4/5 : Implémentation	12
3.13	Étape 5/5 : PR	12
3.14	1. Analyse de l'erreur	12
3.15	2. Hypothèses	12
3.16	3. Investigation	13
3.17	4. Solution	13
3.18	Rapport de Sécurité	13
3.18.1	[RED] Critiques (2)	13
3.18.2	[WARN] Moyens (3)	14
3.18.3	[GREEN] Bonnes pratiques détectées (5)	14
4	Les Hooks	17
4.1	Hooks configurés (8)	17
4.1.1	PreToolUse (3 hooks)	17
4.1.2	PostToolUse (5 hooks)	17
4.2	Exemple en action	17
4.3	Configuration des hooks	17
5	Scripts Utilitaires	19
5.1	Vue d'ensemble	19
5.2	Exemples d'utilisation	19
5.2.1	Installation avec options	19
5.2.2	Validation pour CI/CD	19
5.2.3	Diagnostic complet	19
6	Personnalisation	21
6.1	Fichier CLAUDE.md	21
6.2	Fichier CLAUDE.local.md	21
6.3	Ajouter un agent personnalisé	21
7	Bonnes Pratiques	23
7.1	Les 10 commandements de claude-socle	23
7.1.1	1. Toujours explorer avant de modifier	23
7.1.2	2. Toujours planifier les features complexes	23
7.1.3	3. Utiliser TDD pour le code critique	23
7.1.4	4. Commits atomiques et fréquents	23
7.1.5	5. Review avant merge	23
7.1.6	6. Security audit sur le code sensible	23
7.1.7	7. Documenter les décisions	23
7.1.8	8. Utiliser les templates adaptés	23
7.1.9	9. Valider régulièrement	23
7.1.10	10. Mettre à jour le socle	24
8	Troubleshooting	25
8.1	Problèmes courants	25
8.1.1	"Command not found: claude"	25
8.1.2	"Permission denied" sur les scripts	25
8.1.3	Les hooks ne se déclenchent pas	25
8.1.4	Score de validation < 100%	25

8.1.5	Conflit avec configuration existante	25
8.2	Obtenir de l'aide	25
9	Référence Rapide	26
9.1	Commandes les plus utilisées	26
9.2	Scripts utilitaires	26
9.3	Structure type d'un projet	26

1 Introduction

1.1 Qu'est-ce que claude-socle ?

claude-socle est un template de configuration pour Claude Code qui transforme votre assistant IA en un véritable partenaire de développement. Il fournit :

- **79 agents spécialisés** pour différentes tâches de développement
- **9 skills** pour des comportements automatiques intelligents
- **8 hooks** pour automatiser les vérifications et le formatage
- **8 templates** adaptés à différents langages et frameworks
- **8 scripts utilitaires** pour l'installation et la maintenance

1.2 Pourquoi utiliser claude-socle ?

Sans claude-socle	Avec claude-socle
Prompts répétitifs	Commandes prêtes à l'emploi
Workflow inconsistant	Workflow structuré Explore → Plan → Code → Commit
Risques de sécurité	Hooks de protection intégrés
Configuration manuelle	Installation automatisée

1.3 Prérequis

Avant d'utiliser claude-socle, assurez-vous d'avoir :

- **Claude Code CLI** installé (`npm install -g @anthropic-ai/claude-code`)
- **Git** version 2.0+
- **Node.js** version 18+ (recommandé)
- **jq** pour la validation JSON (optionnel mais recommandé)

2 Installation

2.1 Option 1 : Script d'installation (recommandé)

```
# Cloner le repository
git clone https://github.com/votre-repo/claude-socle.git

# Installer dans un projet existant
./claude-socle/scripts/install.sh /chemin/vers/votre-projet
```

Le script vous guidera à travers les options :

```
=====
Installation Claude Code Configuration
=====

[INFO] Source: /home/user/claude-socle
[INFO] Cible:  /home/user/mon-projet

? Que souhaitez-vous installer?
  1. Configuration complète (recommandé)
  2. Commandes uniquement
  3. Hooks uniquement
  4. Sélection personnalisée
```

2.2 Option 2 : Installation manuelle

```
# Copier la configuration
cp -r claude-socle/.claude votre-projet/
cp claude-socle/CLAUDE.md votre-projet/

# Optionnel : MCP et GitHub Actions
cp claude-socle/.mcp.json votre-projet/
cp -r claude-socle/.github votre-projet/
```

2.3 Option 3 : Création d'un nouveau projet

```
./scripts/new-project.sh
```

Ce script interactif vous guide pour :

1. Choisir le nom du projet
2. Sélectionner le template adapté (React, Node.js, Python, etc.)
3. Détecter automatiquement les technologies existantes
4. Configurer les hooks appropriés

2.4 Vérification de l'installation

```
# Diagnostic complet
```

```
./scripts/doctor.sh /chemin/vers/votre-projet
```

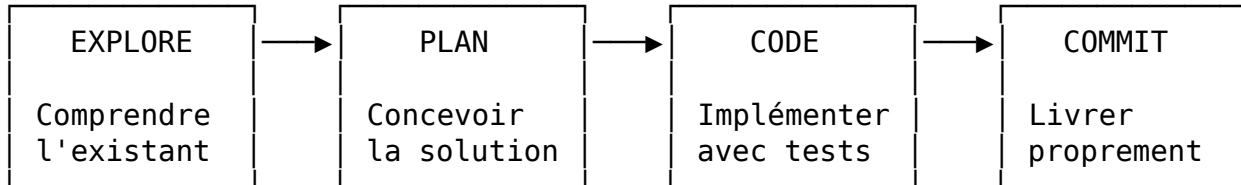
```
# Validation de la configuration
```

```
./scripts/validate.sh /chemin/vers/votre-projet
```

3 Le Workflow Explore → Plan → Code → Commit

3.1 Vue d'ensemble

Le cœur de claude-socle est un workflow en 4 étapes qui garantit un code de qualité :



Règle d'or : Ne jamais coder sans avoir exploré et planifié !

3.2 Étape 1 : EXPLORE

3.2.1 Commande

```
/project:explore [cible]
```

3.2.2 Quand l'utiliser ?

- Avant de modifier du code existant
- Pour comprendre une architecture inconnue
- Pour identifier les dépendances et impacts

3.2.3 Exemple pratique

Utilisateur: `/project:explore le système d'authentification`

Claude: Je vais explorer le système d'authentification...

Architecture identifiée

Fichiers principaux

- ``src/auth/AuthService.ts`` - Service principal (234 lignes)
- ``src/auth/middleware/authMiddleware.ts`` - Middleware Express
- ``src/auth/strategies/`` - Stratégies Passport.js

Technologies utilisées

- Passport.js pour l'authentification
- JWT pour les tokens
- bcrypt pour le hashing

Points d'attention

- Le refresh token n'est pas implémenté
- Pas de rate limiting sur `/login`

Recommandations

Avant toute modification, considérez :

1. L'impact sur les 12 routes protégées
2. La migration des sessions existantes

3.3 Étape 2 : PLAN

3.3.1 Commande

```
/project:plan [fonctionnalité]
```

3.3.2 Quand l'utiliser ?

- Après l'exploration, avant de coder
- Pour des fonctionnalités complexes
- Pour valider une approche avec l'équipe

3.3.3 Exemple pratique

Utilisateur: /project:plan ajouter l'authentification OAuth2 Google

Claude: Je vais planifier l'implémentation d'OAuth2 Google...

Plan d'implémentation

Phase 1 : Configuration (30 min)

1. Ajouter les dépendances
 - `passport-google-oauth2`
 - `@types/passport-google-oauth2`
2. Créer les variables d'environnement
 - `GOOGLE\_CLIENT\_ID`
 - `GOOGLE\_CLIENT\_SECRET`
 - `GOOGLE\_CALLBACK\_URL`

Phase 2 : Backend (2h)

1. Créer `src/auth/strategies/googleStrategy.ts`
2. Ajouter les routes `/auth/google` et `/auth/google/callback`
3. Modifier `AuthService.ts` pour gérer les comptes liés

Phase 3 : Frontend (1h)

1. Ajouter le bouton "Se connecter avec Google"
2. Gérer le callback et le stockage du token

Phase 4 : Tests (1h)

1. Tests unitaires de GoogleStrategy
2. Tests d'intégration du flow complet

Risques identifiés

- Migration des utilisateurs existants
- Gestion des comptes dupliqués (même email)

Souhaitez-vous que je commence l'implémentation ?

3.4 Étape 3 : CODE

3.4.1 Commandes disponibles

Commande	Usage
/project:tdd [feature]	Développement Test-Driven
/project:api [endpoint]	Création d'API
/project:refactor [cible]	Refactoring guidé

3.4.2 Exemple TDD

Utilisateur: /project:tdd GoogleStrategy

Claude: Je démarre le cycle TDD Red-Green-Refactor...

[RED] RED - Écriture du test

```
```typescript
// src/auth/strategies/__tests__/googleStrategy.test.ts
describe('GoogleStrategy', () => {
 it('should create user from Google profile', async () => {
 const profile = {
 id: 'google-123',
 emails: [{ value: 'user@gmail.com' }],
 displayName: 'Test User'
 };

 const user = await GoogleStrategy.validateUser(profile);

 expect(user.email).toBe('user@gmail.com');
 expect(user.provider).toBe('google');
 });
});
```

Test lancé... [X] ÉCHEC (GoogleStrategy n'existe pas)

## 3.5 [GREEN] GREEN - Implémentation minimale

```
// src/auth/strategies/googleStrategy.ts
export class GoogleStrategy {
```

```
static async validateUser(profile: GoogleProfile): Promise<User> {
 return {
 email: profile.emails[0].value,
 provider: 'google',
 providerId: profile.id
 };
}
}
```

Test lancé... [OK] SUCCÈS

### 3.6 [REFACTOR] REFACTOR - Amélioration

[Continue avec l'intégration Passport.js...]

## Étape 4 : COMMIT

### Commande

/project:commit [contexte]

### Ce que fait l'agent

1. Analyse les changements (`git diff`)
2. Génère un message Conventional Commits
3. Vérifie les tests (hook pre-commit)
4. Crée le commit avec co-author

### Exemple

Utilisateur: /project:commit

Claude: J'analyse les changements pour créer le commit...

### 3.7 Changements détectés

#### 3.7.1 Fichiers modifiés (4)

- src/auth/strategies/googleStrategy.ts (+89 lignes)
- src/auth/routes/authRoutes.ts (+24 lignes)
- src/auth/AuthService.ts (+12 lignes)
- .env.example (+3 lignes)

#### 3.7.2 Fichiers de test (2)

- src/auth/strategies/\_\_tests\_\_/googleStrategy.test.ts (+67 lignes)

### 3.8 Message de commit proposé

feat(auth): add Google OAuth2 authentication

- Implement GoogleStrategy with Passport.js
- Add /auth/google and /auth/google/callback routes
- Support account linking for existing users
- Add comprehensive test coverage

Closes #42

Co-Authored-By: Claude <noreply@anthropic.com>

Voulez-vous que je crée ce commit ?

\newpage

# Les Agents par Catégorie

## Workflow Principal (8 agents)

Agent	Commande	Description
Explore	`/project:explore`	Explorer le code existant
Plan	`/project:plan`	Planifier une implémentation
Commit	`/project:commit`	Créer un commit propre
PR	`/project:pr`	Créer une Pull Request
Flow Feature	`/project:flow-feature`	Workflow complet feature
Flow Bugfix	`/project:flow-bugfix`	Workflow correction bug
Flow Release	`/project:flow-release`	Workflow release
Flow Launch	`/project:flow-launch`	Workflow lancement

### Exemple : Flow Feature complet

Utilisateur: /project:flow-feature ajouter le dark mode

Claude: Je lance le workflow complet pour la feature "dark mode"...

### 3.9 Étape 1/5 : Création de la branche

git checkout -b feature/dark-mode [OK] Branche créée

### 3.10 Étape 2/5 : Exploration

[Analyse du système de thèmes existant...]

### 3.11 Étape 3/5 : Planification

[Plan d'implémentation généré...]

### 3.12 Étape 4/5 : Implémentation

[En attente de votre validation du plan...]

### 3.13 Étape 5/5 : PR

[Sera créée après l'implémentation]

## Développement (10 agents)

Agent	Commande	Usage
TDD	`/project:tdd`	Test-Driven Development
Test	`/project:test`	Générer des tests
Debug	`/project:debug`	Débogage méthodique
Refactor	`/project:refactor`	Refactoring guidé
API	`/project:api`	Créer/documenter API
Component	`/project:component`	Créer un composant
Error Handling	`/project:error-handling`	Gestion d'erreurs
API Versioning	`/project:api-versioning`	Versioning d'API
Feature Flag	`/project:feature-flag`	Feature flags
State	`/project:state`	Gestion d'état

### Exemple : Debug

Utilisateur: /project:debug l'erreur "Cannot read property 'user' of undefined"

Claude: Je lance une investigation méthodique...

### 3.14 1. Analyse de l'erreur

Type: TypeError Message: Cannot read property 'user' of undefined Fichier probable: src/middleware/authMiddleware.ts

### 3.15 2. Hypothèses

#### 1. req.session est undefined

- Probabilité: 70%
- Cause: Session middleware non initialisé

#### 2. Race condition

- Probabilité: 20%
- Cause: Accès avant chargement de la session

#### 3. Mauvaise configuration

- Probabilité: 10%
- Cause: Variable d'environnement manquante

### 3.16 3. Investigation

Vérifions l'hypothèse 1...

```
// src/middleware/authMiddleware.ts:23
export const requireAuth = (req, res, next) => {
 if (req.session.user) { // ← Erreur ici
 //...
 }
};
```

### 3.17 4. Solution

```
export const requireAuth = (req, res, next) => {
 if (req.session?.user) { // Optional chaining
 //...
 }
};
```

Voulez-vous que j'applique cette correction ?

## Qualité (8 agents)

Agent	Commande	Usage
Review	`/project:review`	Code review détaillée
Security	`/project:security`	Audit sécurité OWASP
Performance	`/project:perf`	Analyse performance
Ally	`/project:ally`	Audit accessibilité
Lint	`/project:lint`	Analyse statique
Coverage	`/project:coverage`	Couverture de tests
Complexity	`/project:complexity`	Analyse complexité
Dependencies	`/project:deps`	Audit dépendances

### Exemple : Security Audit

Utilisateur: /project:security src/api/

Claude: Je lance un audit de sécurité OWASP Top 10...

### 3.18 Rapport de Sécurité

#### 3.18.1 [RED] Critiques (2)

##### 3.18.1.1 1. Injection SQL - A03:2021 Fichier: src/api/users/userRepository.ts:45

```
// [X] Vulnérable
const query = `SELECT * FROM users WHERE id = ${userId}`;
```

```
// [OK] Correction
const query = 'SELECT * FROM users WHERE id = $1';
await db.query(query, [userId]);
```

### 3.18.1.2 2. Secrets exposés - A02:2021 Fichier: src/api/config.ts:12

```
// [X] Secret en dur
const JWT_SECRET = 'my-super-secret-key';

// [OK] Correction
const JWT_SECRET = process.env.JWT_SECRET;
```

## 3.18.2 [WARN] Moyens (3)

[...]

## 3.18.3 [GREEN] Bonnes pratiques détectées (5)

- [OK] HTTPS forcé en production
- [OK] Headers de sécurité configurés
- [OK] Rate limiting sur /api/auth
- [OK] Validation des entrées avec Joi
- [OK] Logs sans données sensibles

## Operations (16 agents)

Agent	Commande	Usage
Docker	`/project:docker`	Dockeriser
K8s	`/project:k8s`	Kubernetes
CI/CD	`/project:cicd`	Pipelines
Deploy	`/project:deploy`	Déploiement
Monitor	`/project:monitor`	Monitoring
Backup	`/project:backup`	Sauvegardes
Hotfix	`/project:hotfix`	Fix urgent
Release	`/project:release`	Releases
Rollback	`/project:rollback`	Rollback
Scale	`/project:scale`	Scaling
Migrate DB	`/project:migrate-db`	Migration DB
Infra	`/project:infra`	Infrastructure
SSL	`/project:ssl`	Certificats
DNS	`/project:dns`	Configuration DNS
CDN	`/project:cdn`	Configuration CDN
Logs	`/project:logs`	Analyse logs

## Documentation (9 agents)

Agent	Commande	Usage
Doc	`/project:doc`	Documentation
API Doc	`/project:api-doc`	Doc API
README	`/project:readme`	README
Changelog	`/project:changelog`	Changelog
ADR	`/project:adr`	Architecture Decision
Tutorial	`/project:tutorial`	Tutoriels
Diagram	`/project:diagram`	Diagrammes
Onboard	`/project:onboard`	Onboarding
Explain	`/project:explain`	Explications

\newpage

## # Les Skills

Les skills sont des comportements automatiques qui se déclenchent selon le contexte.

### ## Skills disponibles (9)

Skill	Déclencheur	Action
`exploring-codebase`	Exploration de code	Guide l'exploration méthodique
`planning-implementation`	Planification	Structure le plan d'implémentation
`generating-commit-messages`	Commit	Génère des messages Conventional Commits
`creating-pull-requests`	PR	Crée des PR complètes
`reviewing-code`	Review	Effectue des reviews approfondies
`debugging-issues`	Bug	Guide le débogage
`test-driven-development`	TDD	Applique le cycle Red-Green-Refactor
`api-development`	API	Guide la création d'API
`security-audit`	Sécurité	Applique OWASP Top 10

### ## Comment fonctionnent les skills ?

Les skills sont définis dans `.claude/skills/` et contiennent :

1. **\*\*Frontmatter YAML\*\*** : Métadonnées et déclencheurs
2. **\*\*Instructions\*\*** : Guide détaillé pour Claude

### ### Exemple : Skill TDD

```
```yaml
---
name: test-driven-development
description: Développement TDD avec cycle Red-Green-Refactor
allowed-tools:
  - Read
```

```
- Write
- Edit
- Bash
---
```

Test-Driven Development

Cycle obligatoire

[RED] RED

1. Écrire un test qui échoue
2. Le test doit être minimal et ciblé
3. Exécuter pour confirmer l'échec

[GREEN] GREEN

1. Écrire le code MINIMUM pour passer le test
2. Pas d'optimisation prématurée
3. Exécuter pour confirmer le succès

[REFACTOR] REFACTOR

1. Améliorer le code sans changer le comportement
2. Les tests doivent rester verts
3. Appliquer les principes SOLID

[...]

4 Les Hooks

Les hooks automatisent des vérifications et actions à chaque opération.

4.1 Hooks configurés (8)

4.1.1 PreToolUse (3 hooks)

Hook	Action	Comportement
Protection main	Bloque les modifications sur main/master	Bloquant
Tests pre-commit	Exécute les tests avant commit	Bloquant
Détection secrets	Détecte les secrets dans le code	Bloquant

4.1.2 PostToolUse (5 hooks)

Hook	Action	Comportement
Auto-format	Prettier sur fichiers modifiés	Non-bloquant
Type check	Vérifie les types TypeScript	Non-bloquant
ESLint	Vérifie le code	Non-bloquant
npm install	Auto-install après package.json	Non-bloquant
Coverage	Vérifie la couverture après tests	Non-bloquant

4.2 Exemple en action

Utilisateur: Modifie `src/api/users.ts` pour ajouter un endpoint

Claude: [Modifie le fichier...]

```
[>] Hook PostToolUse: Auto-format  
    → prettier --write src/api/users.ts [OK]
```

```
[>] Hook PostToolUse: Type check  
    → tsc --noEmit [OK]
```

```
[>] Hook PostToolUse: ESLint  
    → eslint src/api/users.ts [OK]
```

4.3 Configuration des hooks

Les hooks sont configurés dans `.claude/settings.json` :

```
{
  "hooks": {
    "PreToolUse": [
      {
        "description": "Empêcher modifications sur main",
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "bash -c '[script de vérification]'",
            "onFailure": "block"
          }
        ]
      }
    ]
  }
}
```

5 Scripts Utilitaires

5.1 Vue d'ensemble

Script	Commande	Description
install.sh	./scripts/install.sh [projet]	Installation
update.sh	./scripts/update.sh [projet]	Mise à jour
validate.sh	./scripts/validate.sh [projet]	Validation
new-project.sh	./scripts/new-project.sh	Nouveau projet
uninstall.sh	./scripts/uninstall.sh [projet]	Désinstallation
doctor.sh	./scripts/doctor.sh [projet]	Diagnostic
diff.sh	./scripts/diff.sh [projet]	Comparaison

5.2 Exemples d'utilisation

5.2.1 Installation avec options

```
# Installation interactive (défaut)
./scripts/install.sh ./mon-projet

# Installation silencieuse pour CI/CD
./scripts/install.sh -y ./mon-projet

# Simulation sans modification
./scripts/install.sh --dry-run ./mon-projet

# Installation complète avec hooks et MCP
./scripts/install.sh --all ./mon-projet
```

5.2.2 Validation pour CI/CD

```
# Validation standard
./scripts/validate.sh ./mon-projet

# Sortie JSON pour parsing
./scripts/validate.sh --json ./mon-projet

# Score uniquement
./scripts/validate.sh --score ./mon-projet
# Output: 22/23 (95%)
```

5.2.3 Diagnostic complet

```
./scripts/doctor.sh ./mon-projet
```

Output :

```
=====
Diagnostic Claude Code
=====

1. Environnement système
-----
[✓] Système d'exploitation: Linux
[✓] Shell: zsh
[✓] Bash version: 5.2
[✓] Permissions d'écriture: OK

2. Dépendances
-----
[✓] git: 2.51.0
[✓] jq: 1.8
[✓] Node.js: 20.19.5

[...]

=====
Résumé du diagnostic
=====

✓ Réussis:      25
! Avertissements: 0
x Échoués:      0

[✓] Environnement parfait!
```

6 Personnalisation

6.1 Fichier CLAUDE.md

Le fichier CLAUDE.md à la racine de votre projet contient les instructions spécifiques. Personnalisez-le selon vos besoins :

```
# Mon Projet

## Commandes essentielles
| Commande | Description |
|-----|-----|
| `npm run dev` | Serveur de développement |
| `npm test` | Lancer les tests |

## Conventions de code
- TypeScript strict obligatoire
- Pas de `any`
- Tests pour chaque fonctionnalité

## Architecture
[Décrivez votre architecture...]
```

6.2 Fichier CLAUDE.local.md

Pour les configurations personnelles non versionnées :

```
# Configuration locale

## Mes préférences
- Utiliser des emojis dans les commits
- Commenter en français
- Verbose mode activé

## Contexte
Je travaille sur le module d'authentification cette semaine.
```

6.3 Ajouter un agent personnalisé

Créez un fichier dans .claude/commands/ :

```
# Agent Mon-Agent

## Description
Description de ce que fait l'agent.

## Déclencheurs
- Quand l'utilisateur demande X
- Après Y
```

Instructions

1. Première étape
2. Deuxième étape

Output attendu

Format de sortie souhaité.

7 Bonnes Pratiques

7.1 Les 10 commandements de claude-socle

7.1.1 1. Toujours explorer avant de modifier

[X] "Ajoute une fonction de login"
[OK] "/project:explore le système d'auth" puis "Ajoute..."

7.1.2 2. Toujours planifier les features complexes

[X] "Implémente OAuth2"
[OK] "/project:plan OAuth2" → validation → implémentation

7.1.3 3. Utiliser TDD pour le code critique

[OK] /project:tdd pour les services et utilitaires

7.1.4 4. Commits atomiques et fréquents

[OK] Un commit = une modification logique
[X] Un commit géant avec 50 fichiers

7.1.5 5. Review avant merge

[OK] /project:review avant chaque PR

7.1.6 6. Security audit sur le code sensible

[OK] /project:security sur auth, payments, user data

7.1.7 7. Documenter les décisions

[OK] /project:adr pour les choix architecturaux

7.1.8 8. Utiliser les templates adaptés

[OK] CLAUDE.react.md pour un projet React
[OK] CLAUDE.node-api.md pour une API Node.js

7.1.9 9. Valider régulièrement

```
./scripts/validate.sh . && echo "OK"
```

7.1.10 10. Mettre à jour le socle

```
./scripts/update.sh .
```


8 Troubleshooting

8.1 Problèmes courants

8.1.1 “Command not found: claude”

```
# Installer Claude Code CLI
npm install -g @anthropic-ai/claude-code

# Vérifier l'installation
claude --version
```

8.1.2 “Permission denied” sur les scripts

```
chmod +x ./scripts/*.sh
```

8.1.3 Les hooks ne se déclenchent pas

Vérifiez que les hooks sont dans `settings.json` (pas dans un fichier `hooks.json` séparé).

```
# Vérifier la configuration
jq '.hooks' .claude/settings.json
```

8.1.4 Score de validation < 100%

```
# Identifier les problèmes
./scripts/validate.sh --verbose .
```

8.1.5 Conflit avec configuration existante

```
# Sauvegarder puis réinstaller
./scripts/uninstall.sh --keep-claude-md .
./scripts/install.sh .
```

8.2 Obtenir de l'aide

```
# Aide sur un script
./scripts/install.sh --help

# Diagnostic complet
./scripts/doctor.sh .

# Vérifier les différences avec le socle
./scripts/diff.sh .
```

9 Référence Rapide

9.1 Commandes les plus utilisées

Tâche	Commande
Explorer du code	/project:explore [cible]
Planifier une feature	/project:plan [feature]
Développer en TDD	/project:tdd [feature]
Créer un commit	/project:commit
Créer une PR	/project:pr
Review de code	/project:review [cible]
Audit sécurité	/project:security [cible]
Déboguer	/project:debug [problème]

9.2 Scripts utilitaires

```
./scripts/install.sh [projet]      # Installer
./scripts/update.sh [projet]      # Mettre à jour
./scripts/validate.sh [projet]    # Valider
./scripts/doctor.sh [projet]      # Diagnostiquer
./scripts/diff.sh [projet]        # Comparer
./scripts/uninstall.sh [projet]   # Désinstaller
```

9.3 Structure type d'un projet

```
mon-projet/
├── CLAUDE.md           # Instructions (versionné)
├── CLAUDE.local.md     # Perso (gitignored)
├── .claude/
│   ├── settings.json  # Permissions + hooks
│   ├── commands/      # Agents
│   └── skills/         # Skills
└── ...
```